Harbor Stitch Apparel
2025 operational audit briefing

PRELIMINARY SCREENING • PREPARED 15 OCTOBER 2026

This review uses fictional business records and is an operational screening, not a certified audit opinion. Review indicators identify records that merit corroboration; they do not prove fraud, misconduct, or intent.

1. Schema observed before querying

The PostgreSQL source contains eight public tables. Production is held in **production_batches** (batch, order, line, product, period, produced/good/defective units, fabric used and labor hours), with line names in **production_lines** and fabric standards in **products**. Demand is in **customer_orders**, linked to **customers**. Accounts payable is split across **supplier_invoices**, **supplier_payments** and **suppliers**. The date, amount and identifier fields support bounded 2025 tests without altering the source.

2. Findings and explanations

Monthly production. Recorded production totaled **130,528 units** and equaled recorded ordered units in every month. November was the peak at 13,218 units; February was the low at 9,254. July is the clear quality exception: 343 defects drove the monthly defect rate to **3.59%**, versus approximately 2.49–2.51% in every other month. Follow up the July process records and rework logs.

Defects by production line. The weighted annual defect rate was **2.58%**. Line 2 ran at **2.74%** (1,188 defects), above Lines 3 and 1 at 2.51% and 2.50%. The July follow-up isolates the spike: Line 2 reached **5.83%** (181 defects across 3,107 units), while Lines 1 and 3 each remained at 2.51%. This is a production-control exception, not evidence of wrongdoing.

Duplicate supplier payment screen. No invoice had multiple 2025 payments, and no invoice-level overpayment was found. A weaker but specific review indicator remains: Coastal Textile Mills reference **V01-2025-007** appears on two invoice IDs dated 12 July, both for **$3,595**, and both were paid on consecutive days (6 and 7 August). The review exposure is $3,595—one of the two matching payments. Obtain the two invoice images, purchase orders, receipts and approval trail before concluding whether this was a duplicate or two valid deliveries.

Fabric efficiency. Actual fabric use was **210,809.7 m** against a production standard of **203,848.3 m**: **96.7% weighted efficiency** and 6,961.4 m above standard. Workwear polo had the lowest efficiency at 95.8% (+1,649.3 m). Canvas work jacket had the largest absolute excess at 2,383.4 m despite the best rate, 97.1%, because it used the greatest volume. Review both rate and absolute waste when prioritizing action.

3. Method and scope

All source retrieval used bounded, read-only SELECT queries against data source 237570a0-9428-40e0-841d-11dea515edc0. Tests were restricted to dates from 1 January 2025 through 31 December 2025. Production and defect rates use summed units. Fabric efficiency equals standard metres for actual output divided by actual metres used. Payment tests distinguish invoice-level multiple payments and overpayments from repeated supplier references across separate invoice IDs.

Limits: the analysis relies on recorded fields and does not authenticate invoices, receiving evidence, approvals, bank settlement, inventory movements or management explanations. Exact matches can arise from legitimate recurring or split transactions. Standard fabric rates may not reflect approved substitutions or production complexity.

4. Exact executed SQL

The statements below are also saved as harbor-audit.sql. Comments record the two zero-row payment tests.

```
-- Harbor Stitch Apparel — 2025 preliminary operational screening
-- Source: PostgreSQL data source 237570a0-9428-40e0-841d-11dea515edc0
-- All statements are bounded, read-only SELECT queries executed on 2026-10-15.

-- 1. Monthly production, orders, good units, and defects
WITH monthly_orders AS (
  SELECT date_trunc('month', period)::date AS month,
         SUM(ordered_units)::bigint AS ordered_units
  FROM public.customer_orders
  WHERE period >= DATE '2025-01-01' AND period < DATE '2026-01-01'
  GROUP BY 1
),
monthly_production AS (
  SELECT date_trunc('month', period)::date AS month,
         SUM(produced_units)::bigint AS produced_units,
         SUM(good_units)::bigint AS good_units,
         SUM(defective_units)::bigint AS defective_units
  FROM public.production_batches
  WHERE period >= DATE '2025-01-01' AND period < DATE '2026-01-01'
  GROUP BY 1
)
SELECT COALESCE(p.month, o.month) AS month,
       COALESCE(o.ordered_units, 0) AS ordered_units,
       COALESCE(p.produced_units, 0) AS produced_units,
       COALESCE(p.good_units, 0) AS good_units,
       COALESCE(p.defective_units, 0) AS defective_units,
       ROUND(100.0 * COALESCE(p.defective_units, 0) /
             NULLIF(COALESCE(p.produced_units, 0), 0), 2) AS defect_rate_pct
FROM monthly_production p
FULL OUTER JOIN monthly_orders o USING (month)
ORDER BY month
LIMIT 12;

-- 2. Defect rates by production line, compared with the weighted overall rate
SELECT l.line_id,
       l.line_name,
       SUM(b.produced_units)::bigint AS produced_units,
       SUM(b.defective_units)::bigint AS defective_units,
       ROUND(100.0 * SUM(b.defective_units) / NULLIF(SUM(b.produced_units), 0), 2) AS defect_rate_pct,
       ROUND(100.0 * SUM(SUM(b.defective_units)) OVER () /
             NULLIF(SUM(SUM(b.produced_units)) OVER (), 0), 2) AS overall_defect_rate_pct
FROM public.production_batches b
JOIN public.production_lines l ON l.line_id = b.line_id
WHERE b.period >= DATE '2025-01-01' AND b.period < DATE '2026-01-01'
GROUP BY l.line_id, l.line_name
ORDER BY defect_rate_pct DESC, l.line_id
LIMIT 50;

-- 3. Strong duplicate-payment test: multiple payments exceeding one invoice
-- Result: zero rows.
SELECT i.invoice_id,
       s.supplier_name,
       i.supplier_reference,
       i.invoice_date,
       MAX(i.amount) AS invoice_amount,
       COUNT(p.payment_id)::int AS payment_count,
       SUM(p.amount) AS total_paid,
       SUM(p.amount) - MAX(i.amount) AS excess_paid
FROM public.supplier_invoices i
JOIN public.supplier_payments p ON p.invoice_id = i.invoice_id
JOIN public.suppliers s ON s.supplier_id = i.supplier_id
WHERE i.invoice_date >= DATE '2025-01-01' AND i.invoice_date < DATE '2026-01-01'
  AND p.payment_date >= DATE '2025-01-01' AND p.payment_date < DATE '2026-01-01'
GROUP BY i.invoice_id, s.supplier_name, i.supplier_reference, i.invoice_date
HAVING COUNT(p.payment_id) > 1
   AND SUM(p.amount) > MAX(i.amount)
ORDER BY excess_paid DESC, i.invoice_id
LIMIT 100;

-- 4. Secondary test: multiple payments against a single invoice
-- Result: zero rows.
SELECT i.invoice_id,
       s.supplier_name,
       i.supplier_reference,
       i.invoice_date,
       MAX(i.amount) AS invoice_amount,
       COUNT(p.payment_id)::int AS payment_count,
       SUM(p.amount) AS total_paid,
       SUM(p.amount) - MAX(i.amount) AS variance_to_invoice
FROM public.supplier_invoices i
JOIN public.supplier_payments p ON p.invoice_id = i.invoice_id
JOIN public.suppliers s ON s.supplier_id = i.supplier_id
WHERE i.invoice_date >= DATE '2025-01-01' AND i.invoice_date < DATE '2026-01-01'
  AND p.payment_date >= DATE '2025-01-01' AND p.payment_date < DATE '2026-01-01'
GROUP BY i.invoice_id, s.supplier_name, i.supplier_reference, i.invoice_date
HAVING COUNT(p.payment_id) > 1
ORDER BY payment_count DESC, ABS(SUM(p.amount) - MAX(i.amount)) DESC, i.invoice_id
LIMIT 100;

-- 5. Repeated supplier references across distinct invoices
SELECT s.supplier_name,
```

```
       i.supplier_reference,
       COUNT(DISTINCT i.invoice_id)::int AS invoice_count,
       SUM(i.amount) AS invoiced_total,
       COUNT(DISTINCT p.payment_id)::int AS payment_count,
       SUM(p.amount) AS paid_total
FROM public.supplier_invoices i
JOIN public.suppliers s ON s.supplier_id = i.supplier_id
LEFT JOIN public.supplier_payments p
  ON p.invoice_id = i.invoice_id
 AND p.payment_date >= DATE '2025-01-01' AND p.payment_date < DATE '2026-01-01'
WHERE i.invoice_date >= DATE '2025-01-01' AND i.invoice_date < DATE '2026-01-01'
GROUP BY s.supplier_name, i.supplier_reference
HAVING COUNT(DISTINCT i.invoice_id) > 1
ORDER BY invoiced_total DESC, s.supplier_name, i.supplier_reference
LIMIT 100;

-- 6. Detail behind the repeated-reference indicator returned by query 5
SELECT i.invoice_id,
       s.supplier_name,
       i.supplier_reference,
       i.invoice_date,
       i.amount AS invoice_amount,
       p.payment_id,
       p.payment_date,
       p.amount AS payment_amount
FROM public.supplier_invoices i
JOIN public.suppliers s ON s.supplier_id = i.supplier_id
LEFT JOIN public.supplier_payments p
  ON p.invoice_id = i.invoice_id
 AND p.payment_date >= DATE '2025-01-01' AND p.payment_date < DATE '2026-01-01'
WHERE i.invoice_date >= DATE '2025-01-01' AND i.invoice_date < DATE '2026-01-01'
  AND s.supplier_name = 'Coastal Textile Mills'
  AND i.supplier_reference = 'V01-2025-007'
ORDER BY i.invoice_date, i.invoice_id, p.payment_date, p.payment_id
LIMIT 20;

-- 7. Fabric efficiency by product style
SELECT p.product_id,
       p.style_name,
       SUM(b.produced_units)::bigint AS produced_units,
       ROUND(SUM(b.fabric_used_m), 1) AS actual_fabric_m,
       ROUND(SUM(b.produced_units * p.standard_fabric_m), 1) AS standard_fabric_m,
       ROUND(SUM(b.fabric_used_m) - SUM(b.produced_units * p.standard_fabric_m), 1) AS excess_fabric_m,
       ROUND(100.0 * SUM(b.produced_units * p.standard_fabric_m) /
             NULLIF(SUM(b.fabric_used_m), 0), 1) AS fabric_efficiency_pct
FROM public.production_batches b
JOIN public.products p ON p.product_id = b.product_id
WHERE b.period >= DATE '2025-01-01' AND b.period < DATE '2026-01-01'
GROUP BY p.product_id, p.style_name
ORDER BY fabric_efficiency_pct ASC, p.product_id
LIMIT 100;

-- 8. July line-level follow-up for the monthly defect spike
SELECT l.line_id,
       l.line_name,
       SUM(b.produced_units)::bigint AS produced_units,
       SUM(b.defective_units)::bigint AS defective_units,
       ROUND(100.0 * SUM(b.defective_units) / NULLIF(SUM(b.produced_units), 0), 2) AS defect_rate_pct
FROM public.production_batches b
JOIN public.production_lines l ON l.line_id = b.line_id
WHERE b.period >= DATE '2025-07-01' AND b.period < DATE '2025-08-01'
GROUP BY l.line_id, l.line_name
ORDER BY defect_rate_pct DESC, l.line_id
LIMIT 20;
```